

Curso Inicial Front-End

Clase 18: Eventos en el DOM

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

Una web sin interactividad es solo un papel digital. En esta clase aprenderemos a escuchar al usuario mediante los **Eventos**: clicks, envíos de formulario y movimientos del mouse. Además, descubriremos cómo hacer que los datos "sobrevivan" al cerrar el navegador usando **LocalStorage** y **SessionStorage**, permitiendo crear carritos de compra y sesiones persistentes.



Objetivos

- Dominar el uso de `addEventListener` para capturar acciones del usuario.
- Comprender el flujo de los formularios y la importancia de `preventDefault()`.
- Implementar persistencia de datos mediante el uso profesional de Web Storage.



Bloques Temáticos

- Tema 1: Manejo de la interactividad: addEventListener.
 - El Sistema de Alarmas de la Web
 - El Objeto `event` (La Caja Negra del Avión)
 - localStorage: El "Guardar Partida" del Navegador
- Tema 2: Captura y validación de datos en formularios.

Ya sabemos cómo atrapar botones de la pantalla (DOM) usando **JavaScript**. Pero hasta ahora, esos botones son "sordos". Si el usuario hace clic, nada sucede. Hoy aprenderemos a instalar "micrófonos" en nuestra página web. Le enseñaremos a **JavaScript** a quedarse escuchando pacientemente a que el usuario haga algo (como escribir o hacer clic) para reaccionar en el acto.

 **Recurso Oficial:** [MDN Web Docs - Introducción a Eventos](https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Eventos)

Tema 1: Manejo de la interactividad: `addEventListener`.

¿Qué es un Evento?

Un **Evento** es una señal que el navegador emite cuando algo sucede en la página (el usuario hizo clic, terminó de cargar la web, presionó una tecla). Como programadores, nosotros "escuchamos" esos eventos para ejecutar una respuesta lógica.

Catálogo de Eventos Comunes

Evento	Cuándo ocurre	Uso común
<code>click</code>	Al presionar y soltar un elemento.	Botones, enlaces.
<code>submit</code>	Al intentar enviar un <code><form></code> .	Validación antes del envío.
<code>change</code>	Al cambiar el valor de un input.	Selects, checkboxes.
<code>input</code>	Cada vez que se escribe una letra.	Buscadores en tiempo real.
<code>mouseenter</code>	Al pasar el mouse por encima.	Menús desplegables.
<code>keydown</code>	Al presionar una tecla.	Atajos de teclado.
<code>DOMContentLoaded</code>	Cuando el HTML terminó de cargar.	Iniciar el script de forma segura.

1. El Sistema de Alarmas de la Web

Para que un botón reaccione, usamos un método vital llamado `addEventListener()` (Escuchador de Eventos). Imagina que le estás instalando un sensor láser a una puerta.

Mapeo de Sintaxis (Desglosando la magia):

```
/* 1. Atrapamos el botón en el DOM */
const miBoton = document.querySelector("#btn-comprar");

/* 2. Le instalamos el sensor */
// Parámetro 1: El nombre de la acción que estamos esperando escuchar (ej:
"click").
// Parámetro 2: La Función que se disparará automáticamente cuando ocurra
la acción.
miBoton.addEventListener("click", () => {
    console.log("¡El usuario acaba de hacer clic!");
});
```

2. El Objeto `event` (La Caja Negra del Avión)

Cuando el usuario hace clic en el botón, JavaScript no solo ejecuta tu función, sino que te regala un paquete secreto de información sobre el "accidente". Este paquete se atrapa poniendo un parámetro en tu función, generalmente llamado `e` o `event`.

```
/* Observa la 'e' dentro de los paréntesis de la flecha */
document.addEventListener("click", (e) => {
    // La propiedad 'target' te dice el elemento exacto que recibió el
    impacto físico del clic.
    console.log("Acabas de hacer clic en:", e.target);
});
```

El Policía de Tránsito: `e.preventDefault()`

Esto es crucial. Los formularios (`<form>`) en la web antigua (año 1995) fueron diseñados para **recargar la página por completo** cada vez que apretabas el botón de "Enviar". Hoy en día, las páginas modernas (como Facebook o Gmail) NUNCA se recargan al comentar o enviar datos. Para frenar ese comportamiento obsoleto, usamos al policía:

```
formulario.addEventListener("submit", (e) => {  
  // Frena el tiempo. Le dice al navegador: "No te recargues, yo me  
  encargo desde aquí".  
  e.preventDefault();  
  console.log("Formulario capturado por JS de forma segura");  
});
```

🛑 **Checkpoint de Comprensión:** Has diseñado una hermosa pantalla de inicio de sesión. Tienes tu etiqueta `<form>` y tu botón "Ingresar". Escribes tu código **JS**, rellenas el usuario, presionas el botón, y notas que por una fracción de segundo aparece un parpadeo blanco y todos tus datos desaparecen de los inputs sin dejar rastro en la consola. Sabiendo cómo actúan los formularios por defecto, ¿qué línea exacta de código olvidaste escribir dentro de tu evento `submit`?

3. localStorage: El "Guardar Partida" del Navegador

Imagina que un usuario eligió el "Modo Oscuro" de tu página. Si cierra la pestaña y vuelve a entrar mañana, ¿cómo sabe tu **HTML** que quería el fondo negro? La memoria de las variables normales se destruye al cerrar la pestaña. Para guardar datos permanentes usamos la **API de Web Storage**.

- **localStorage (El Disco Duro):** Guarda datos para siempre, hasta que el usuario borre el caché de su computadora.

- **sessionStorage (La Memoria Corta):** Guarda datos, pero se autodestruye violentamente en el milisegundo en que el usuario cierra la pestaña.

Persistencia: LocalStorage vs. SessionStorage

Ambos sirven para guardar datos en el navegador del usuario (como el token de login o el modo oscuro).

Característica	localStorage	sessionStorage
Persistencia	Permanente (hasta que se borre manual).	Solo mientras la pestaña esté abierta.
Cierre de navegador	Los datos sobreviven.	Los datos se borran automáticamente.
Uso común	Preferencias de usuario, carritos.	Datos temporales, formularios largos.

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>